로봇 밸류체인 · AI 에이전트 실무

로봇 셀 SI 제안서 멀티에이전트

클로드 코드 서브에이전트 6종으로 RFQ에서 제안서까지

가상 사례 — CNC 4대 머신텐딩 자동화

요구 분석 → 셀 설계·사이클타임(병렬) → 경제성 → 제안서 → 독립 검수
숫자는 스크립트로 계산하고, 검수자가 다시 돌려 대조한다

이 덱이 전하는 세 가지 메시지

한 사람의 프리세일즈 업무를 여섯 역할로 나눠 맡긴다

1

업무를 역할로 쪼개면 에이전트 팀이 된다

요구 분석·설계·계산·재무·작성·검수는 실제 SI 회사의 분업 구조와 같다

2

에이전트끼리는 파일로만 일을 넘긴다

01~06 산출 파일이 계약서 역할을 해 어느 숫자가 어디서 왔는지 추적된다

3

숫자는 LLM이 아니라 스크립트가 만든다

사이클타임·NPV는 Python으로 계산하고, 독립 검수자가 재실행해 대조한다

결과: 가상 RFQ 한 건 → 추천안·투자비 3.65억·회수 1.71년의 제안서 + 검수 기록

로봇 SI는 로봇을 공정에 맞춰 붙이는 사업이다

SI = System Integration. 로봇 제조사와 최종 수요 기업 사이에 위치한다

밸류체인에서의 위치

- 부품: 감속기·서보모터·센서·그리퍼
- 로봇 제조: 산업용·협동로봇 본체
- SI: 셀 설계·제작·설치·티칭
- 수요 기업: 자동차·전자·물류 공장

제안서가 답해야 할 질문

- 목표 생산량을 맞추는가 (CT)
- 얼마를 쓰고 언제 회수하는가 (ROI)
- 공장에서 안전하게 돌아가는가
- 언제 양산에 들어가는가 (일정)

서브에이전트는 역할별로 분리된 작업 세션이다

클로드 코드에서 .claude/agents/*.md 파일 하나가 에이전트 하나다

구분	메인 세션(오케스트레이터)	서브에이전트
하는 일	일을 나누고 순서를 정하고 검수	자기 단계 하나만 수행
정의 위치	CLAUDE.md	.claude/agents/이름.md
맥락	전체 흐름	자기 입력 파일만 읽음
도구 권한	전체	md 파일 머리말에서 제한

왜 나누는가

- 맥락이 짧아져 실수가 줄고, 단계별로 모델·권한을 다르게 줄 수 있다

01

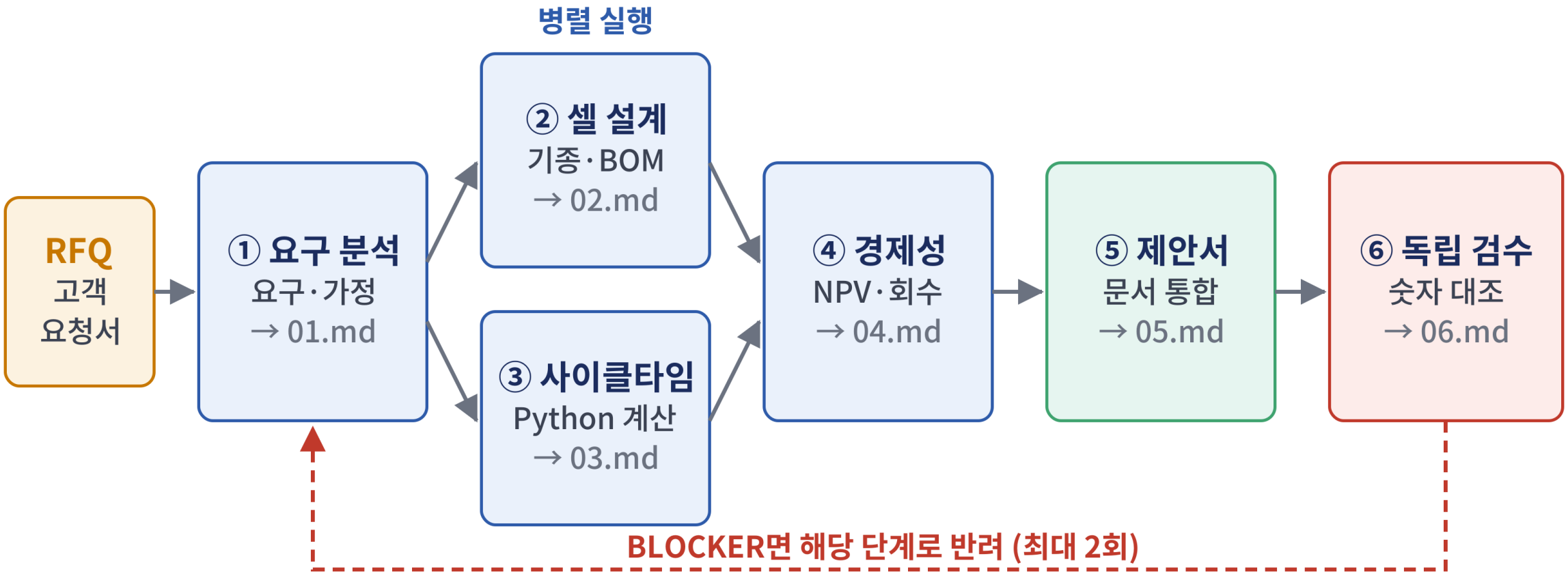
PART 1 · 시스템 설계

에이전트 6종과 파이프라인

무엇을 누구에게 맡기고, 어떻게 넘기는가

파이프라인은 여섯 단계와 반려 루프로 이뤄진다

설계(②)와 사이클타임(③)은 서로 독립이라 동시에 실행한다



에이전트마다 입력·출력·도구가 정해져 있다

model 필드로 설계·작성은 opus, 계산·정리는 sonnet을 쓴다

에이전트	하는 일	출력	도구
requirements-analyst	RFQ → 요구·가정 표	01_requirements	Read·Write
cell-designer	기종·레이아웃·BOM·안전	02_cell_design	+ 웹 검색
cycle-time-engineer	동작 분해 → CT 계산	03_cycle_time	+ Bash
roi-analyst	NPV·IRR·민감도	04_roi	+ Bash
proposal-writer	01~04 통합 제안서	05_proposal	Read·Write
reviewer	숫자 대조·누락 점검	06_review	+ Bash

네 가지 원칙이 결과의 신뢰도를 만든다

모두 CLAUDE.md에 오케스트레이터 규칙으로 적어 둔다

파일이 계약서

handoff

앞 단계 파일만 읽는다.
대화로 넘기지 않는다

01 → 02 → … → 06

숫자는 스크립트

tools/*.py

암산한 숫자는 받지 않는다

cycle_time.py · roi.py

가정에 ID

A-01 …

RFQ에 없는 값은 ID를
붙여 끝까지 인용

A-07 증산 판매 50%

독립 검수

reviewer

작성 과정을 안 본
검수자가 반려·승인

BLOCKER → 재작업

프로젝트는 폴더 하나와 마크다운 파일로 구성된다

에이전트 정의 = 머리말(이름·설명·도구·모델) + 역할 지시문

[출력] 프로젝트 구조

```
CLAUDE.md          ← 오케스트레이터
.claude/agents/    ← 에이전트 6종
  requirements-analyst.md
  cell-designer.md
  cycle-time-engineer.md
  roi-analyst.md
  proposal-writer.md
  reviewer.md
.claude/commands/si-proposal.md
tools/  cycle_time.py  roi.py
input/  rfq_sample.md
output/ 01~06 산출물
```

[출력] .claude/agents/reviewer.md

```
---
name: reviewer
description: 제안서를 RFQ·중간 산출물과
  대조해 숫자 불일치·누락·안전 문제를 찾는다
tools: Read, Write, Bash
model: opus
---
너는 제안 검수 위원이다. 작성 과정에
참여하지 않았다.
## 점검 항목
1. 추적성  2. 숫자 일치  3. 가반하중
4. 안전    5. 과장
```

설정 파일은 직접 쓰지 않고 메타프롬프트로 만든다

[입력] 원하는 팀 구성을 설명 → [출력] Claude가 폴더·파일 일체를 생성

[입력] 프롬프트 · Claude Code

```
로봇 셀 SI 제안서를 만드는 멀티에이전트
프로젝트를 이 폴더에 만들어줘.

- 에이전트 6종을 .claude/agents/ 에:
  요구분석, 셀설계, 사이클타임, 경제성,
  제안서 작성, 독립 검수
- 각 단계는 output/ 파일로만 넘긴다
- 사이클타임·NPV는 tools/ 의 Python
  스크립트로 계산한다
- RFQ에 없는 값은 가정 ID(A-01...)로 관리
- 검수에서 BLOCKER면 그 단계로 되돌린다
- /si-proposal 슬래시 명령도 만들어줘
```

[출력] Claude가 생성

```
CLAUDE.md
.claude/agents/*.md (6개)
.claude/commands/si-proposal.md
tools/cycle_time.py · roi.py
```

사람의 몫

- 생성된 파일을 읽고 고칠 점을 다시 지시
- 샘플 RFQ로 한 번 돌려 본다

실행은 명령 한 줄이면 된다

슬래시 명령 파일 자체가 프롬프트다. 자연어로 같은 지시를 해도 된다

[입력] 명령어 · Claude Code에 입력

```
/si-proposal input/rfq_sample.md
```

[입력] 프롬프트 · Claude Code (같은 효과)

```
input/rfq_sample.md 로 제안서 파이프라인을 돌려줘
```

[출력] 결과 · 오케스트레이터 최종 보고 (작성 예시)

```
① 추천안    B안 협동로봇 + 주행축 + 영역 스캐너 (CNC 4대 담당)
② 경제성    투자비 3.65억 원 · 회수기간 1.71년 (증산 미판매 시 3.65년)
③ 확인      증산분 판매 가능 여부 · 셀 옆 통행 빈도 · CNC 도어 개조 지원

생성 파일   output/01_requirements.md ... 06_review.md
검수 기록   1차 반려(단계 2·4) → 2차 승인
```

02

PART 2 · 샘플 실행

가상 RFQ로 돌린 결과

CNC 4대 머신텐딩 — 요구에서 검수까지

CASE

고객은 사람 손 대신 로봇으로 CNC 4대를 돌리려 한다

가상 고객사 · 자동차 브레이크 캘리퍼 브래킷 가공 라인

4대	54→70	4.5억	3.0 m
CNC 머시닝센터 가공 168초/개	시간당 처리량 현재 → 목표	예산 상한 (VAT 별도)	작업자 통로 셀 옆 통행 유지

① 요구 분석이 짚은 쟁점

- 통로를 막지 않아야 한다는 조건(RFQ §4)이 로봇 방식 선택을 가른다
- 증산분을 팔 수 있는지는 미확인 → 가정 A-07로 두고 고객 확인 질문에 올림

처리량은 설비와 로봇 중 느린 쪽이 정한다

T_door = 도어가 열려 설비가 멈춘 시간, T_off = 설비 가공 중 로봇 단독 작업

$$T_{\text{eff}} = \max\left(\underbrace{T_{\text{CNC}} + T_{\text{door}}}_{\text{machine-bound}},\ \underbrace{n\,(T_{\text{door}} + T_{\text{off}} + T_{\text{move}})}_{\text{robot-bound}} \right)$$

$$Q = \frac{n \cdot 3600}{T_{\text{eff}}} \times A \qquad Q_B = \frac{4 \cdot 3600}{178.8} \times 0.92 = 74.1\ \text{parts/h}$$

계산 결과 (B1)

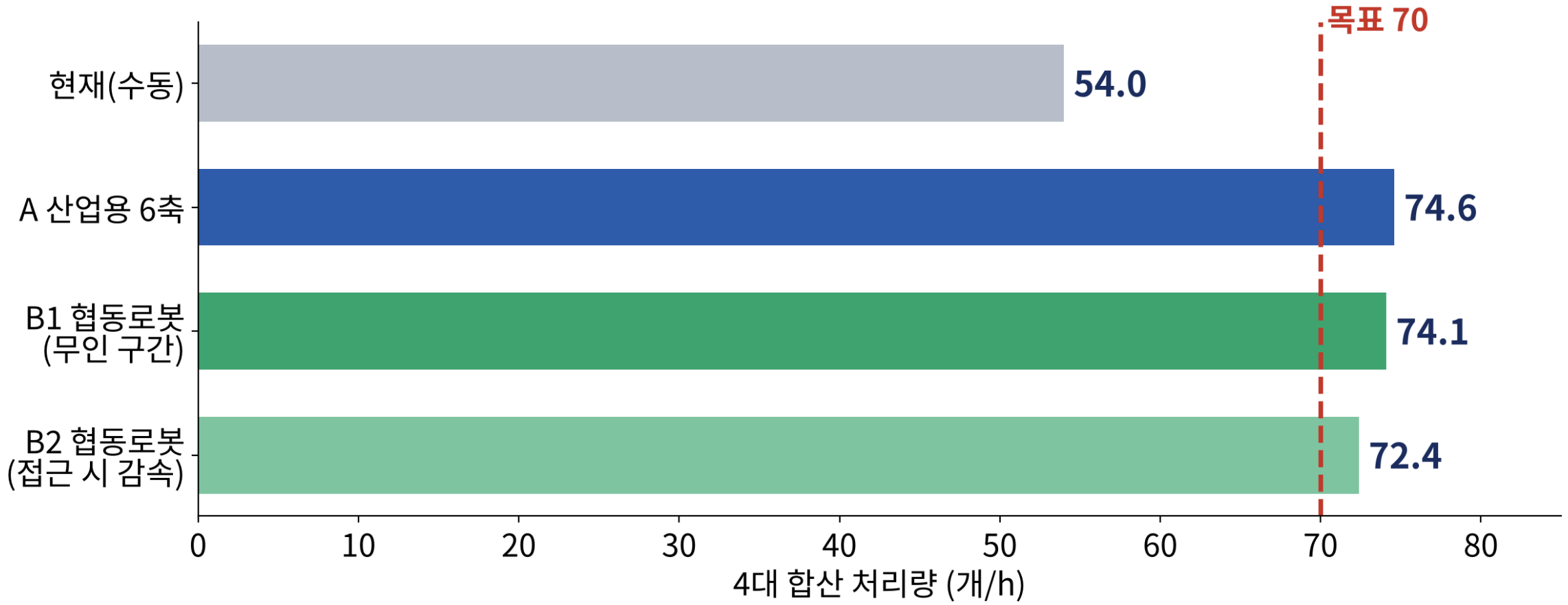
- 설비 측 168 + 10.8 = 178.8초
- 로봇 측 4 × 19.1 = 76.4초

해석

- 병목은 설비 — 로봇은 57% 놀고 있다
- 로봇 속도는 처리량에 거의 영향 없음

세 시나리오 모두 목표 70개를 넘는다

증가분은 대기 손실 제거(가동률 72%→92%)에서 나온다. 작업자 접근으로 계속 감속해도 72.4개



두 안의 성능은 같고 통로 조건이 승부를 가른다

② 셀 설계 에이전트가 낸 대안 비교. 가반하중 = (부품 3.2 + 그리퍼 4.0) × 1.3

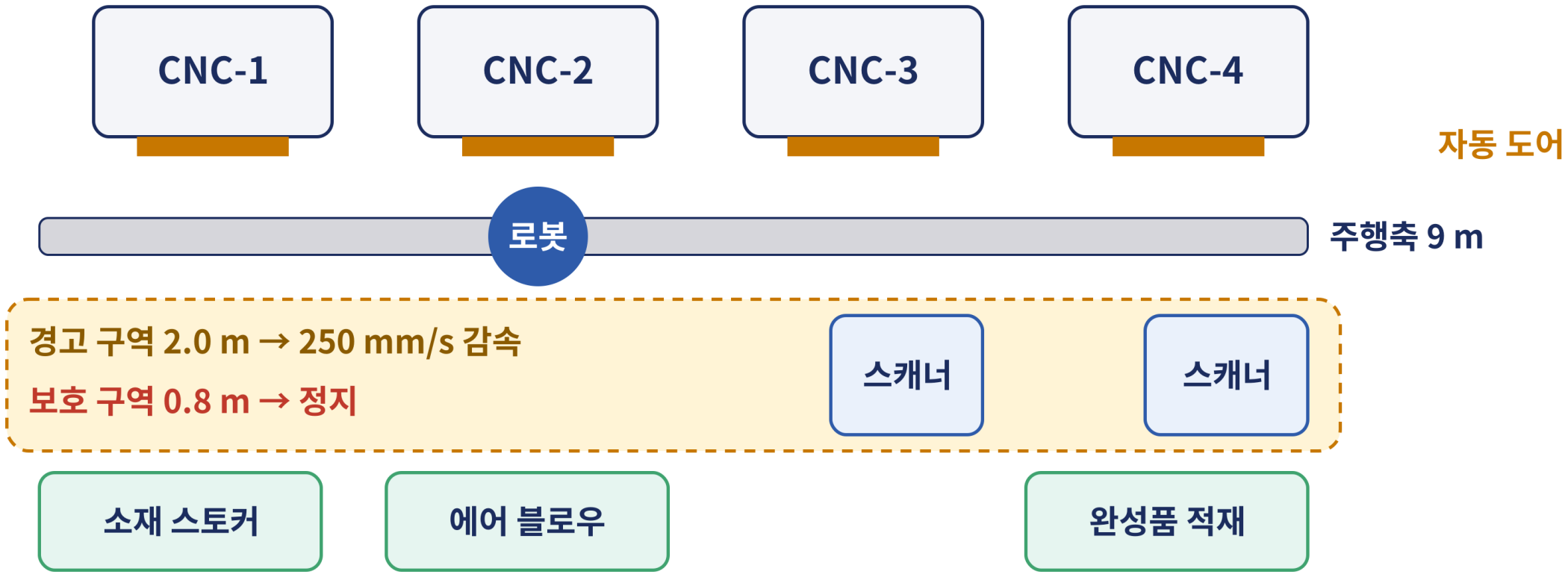
항목	A안 산업용 6축 + 펜스	B안 협동로봇 + 스캐너
가반하중	20 kg급 ≥ 9.4 kg	12 kg급 ≥ 9.4 kg (여유 작음)
처리량	74.6개/h	74.1개/h
투자비	3.90억	3.65억
통로 폭	1.8 m	3.0 m 유지

추천

- B안 — RFQ §4 '셀 옆 통행' 조건을 지키는 유일한 안

B안은 펜스 대신 스캐너로 사람을 감지한다

속도·거리 감시(SSM, ISO/TS 15066) — 접근하면 감속, 더 가까우면 정지



작업자 통로 3.0 m — 펜스 없이 그대로 유지

B안은 1.7년 안에 투자비를 회수한다

할인율 7%, 7년. L=절감 인원, w=1인 인건비, ΔN=증산 판매량, m=개당 공헌이익, M=유지보수

$$\mathrm{NPV} = -I_0 + \sum_{t=1}^{7} \frac{CF}{(1+r)^t}, \qquad \mathrm{Payback} = \frac{I_0}{CF}$$

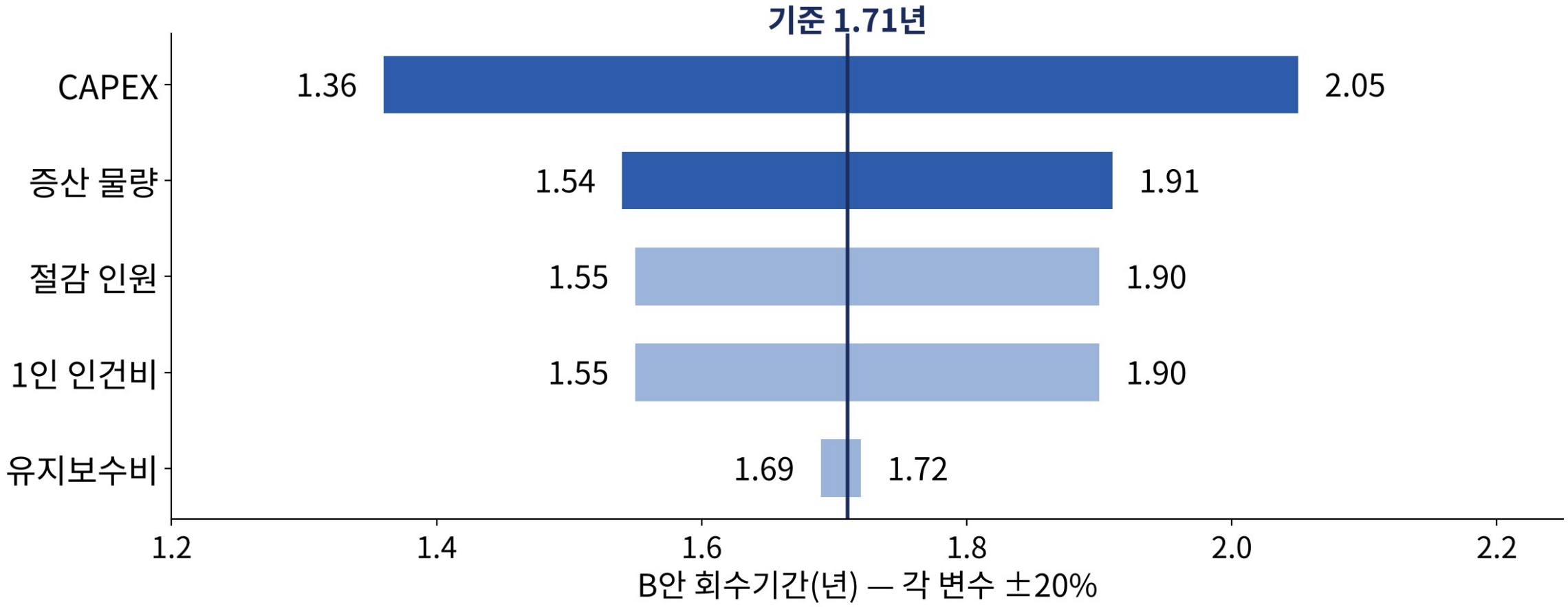
$$CF = \underbrace{L \cdot w}_{\text{labor}} + \underbrace{\Delta N \cdot m}_{\text{extra output}} - M$$

대안	투자비	연 순효과	NPV	IRR	회수
A안 산업용	3.90억	2.18억	7.85억	53.1%	1.79년
B안 협동로봇	3.65억	2.14억	7.88억	56.0%	1.71년
B안 증산 미판매	3.65억	1.00억	1.74억	19.5%	3.65년

증산분을 못 팔아도 NPV는 양(+) — 회수기간만 두 배로 늘어난다

±20% 흔들어도 회수기간은 2.1년 안이다

CAPEX가 가장 크게 흔든다. 진짜 갈림길은 ±20%가 아니라 증산분 판매 여부(1.71년 ↔ 3.65년)



독립 검수는 과장과 누락을 잡아 반려한다

검수 루프 작성 예시 — ⑥ reviewer는 작성 과정을 보지 않고 RFQ·산출물·스크립트만 대조한다

차수	지적	등급	조치
1차	'협동로봇이라 펜스 불필요' 단정	BLOCKER	② 재실행: 위험성 평가 문구·예비비
1차	증산 미판매 보수안 누락	BLOCKER	④ 재실행: 보수 시나리오 추가
1차	설 연휴 미반영 일정	MINOR	⑤ 반영
2차	상위 기종 단가 추정치 단정	MINOR	⑤ '견적 확인 필요'로 수정 → 승인

교훈 검수 에이전트는 '작성자와 다른 눈'일 때 효과가 난다 — 같은 세션이 쓰고 검수하면 놓친다

03

PART 3 · 확장

다른 로봇 업무로 넓히기

같은 골격을 공정·업무만 바꿔 재사용한다

입력 RFQ와 계산 도구만 바꾸면 다른 업무가 된다

에이전트 골격(분석 → 설계·계산 → 재무 → 작성 → 검수)은 그대로 둔다

업무	바꿀 부분	계산 도구
팔레타이징·용접 셀	동작 분해 예시	cycle_time.py 재사용
BOM 원가 절감(VE)	설계 → 대체 소싱 탐색	원가 모델
품질 8D 리포트	설계 → 근본원인 분석	불량 통계
공급망 리스크	RFQ → 뉴스·공급사 DB	노출도 점수

사람이 계속 쥐는 것

- 고객 확인 질문의 답, 최종 단가 견적, 안전 위험성 평가 서명

RFQ 유형이 다르면 바꾸는 범위가 달라진다

맨 앞에서 유형부터 판정한다 — 처음 보는 유형도 AI가 해법을 만들고 사람은 결정한다

RFQ 유형	예	바꿀 부분
같은 유형, 조건만 다름	CNC 6대 · 다른 부품	없음 — 그대로 실행
비슷한 공정	팔레타이징 · 용접 셀	스킬 공정 절 추가 · 도구 재사용
다른 업무	원가 절감(VE) · 품질 8D	설계·도구 교체 → 새 팀 구성
처음 보는 유형	판정이 안 서는 요청	AI가 설계 → 사람이 결정

원칙 — 바꾸는 범위는 작은 것부터

- 입력 → 스킬 → 도구 → 팀 순서로 넓힌다 · 사내 데이터는 전제가 아니라 보정 수단

Day6 course-forge와 두 가지로 이어진다

course-forge = 원본 → 승인 → 검증 → 스킬. 같은 틀로 업무 팀도, 교육 강의도 만든다

① 틀을 빌린다 — si-forge

- AI가 원리·검색으로 처음부터 설계
- 독립 재계산·범위 점검 · 사내 데이터로 보정
- 사람은 승인 세 번 · 최종 견적 · 안전 서명

② 산출물을 잇는다 — learn-si-*

- 등록 유형의 SKILL.md·대조표를 course-forge 원본으로
- 검증 운전 RFQ = 실습, 대조표 숫자 = 정답
- 검수 반려 사례 = 판단형 퀴즈 → 신입 교육

공통 원리: AI가 해법까지 만들고, 숫자는 실행 결과에서만, 사람은 결정과 감사

사내 기준·견적이 담기므로 learn-si-* 강의는 회사 승인 아래 비공개 저장소에 둔다

엔지니어의 시간은 문서 작업에서 판단으로 옮겨 간다

프리세일즈 업무를 단계별로 나눠, 누가 무엇을 맡는지 다시 그린 표

업무	지금	도입 후 에이전트	엔지니어의 몫
RFQ 정리	직접 읽고 표로 정리	요구·가정 표 초안	고객 확인 질문 답 받기
CT·ROI 계산	엑셀 수작업	스크립트로 계산	가정값 타당성 판단
BOM·단가	단가표 뒤져 입력	단가 DB에서 범위 적용	최종 견적 확정
제안서 작성	매번 새로 작성	서식대로 초안	추천안·논리 검토
검수	바쁘면 생략	독립 검수·숫자 대조	반려 사유 판단

RFQ 응답이 빨라지고, 엔지니어는 현장 실사·고객 대화·설계 판단에 시간을 쓴다

선배의 머릿속 기준이 스킬 파일로 회사 자산이 된다

스킬(skill) = 여러 에이전트가 함께 펼쳐 보는 업무 기준서. 서브에이전트는 '누가', 스킬은 '무엇을 기준으로'

이렇게 바뀐다

- 말로 전하던 기준 → 파일 한 곳
- 기준 변경 → 스킬만 수정
- 담당자 교체 → 기준은 남음
- 신입 교육 자료로 재사용

[출력] .claude/skills/si-design-standards/SKILL.md

```
## 1. 로봇 선정
가반하중 = (부품 + 그리퍼) × 1.3 이상
여유 1.3~1.5배 → '여유 작음' 표시

## 2. 안전 규격 적용
사람 접근 시 ISO/TS 15066 적용 여부를
위험성 평가로 결정한다
금지 표현: '협동로봇이라 펜스 불필요'

## 3. 방호 방식 선택
통로 확보가 요구 → 스캐너 + SSM
```

AI가 맡는 일과 사람이 끝까지 쥐는 일은 나뉜다

코딩을 몰라도 된다 — 기존 엑셀 계산표·단가표를 메타프롬프트로 스크립트와 스킬로 옮긴다

에이전트에게 맡긴다

- 반복 문서·표·계산
- 숫자 대조와 누락 점검
- 서식 맞추기

사람이 끝까지 쥔다

- 현장 실측·고객 관계
- 최종 단가 견적
- 위험성 평가 서명

교육 목표: 도구 사용법이 아니라 내 업무를 역할(에이전트)과 기준(스킬)으로 쪼개는 능력

한계: 결과 품질은 단가 DB와 설계 기준을 얼마나 잘 적어 넣느냐에 좌우된다

토론 질문

자기 회사 업무에 대입해 답한다

Q1	우리 팀 업무 중 '파일로 넘길 수 있는' 단계는 어디까지인가	**분업 설계**
Q2	LLM에 맡기면 안 되는 숫자는 무엇이고, 어떤 스크립트로 대체할까	**계산 신뢰**
Q3	검수 에이전트가 반려해야 할 BLOCKER 기준을 정해 보라	**검수 기준**
Q4	제안서에서 끝까지 사람이 서명해야 하는 항목은 무엇인가	**책임 경계**

종합

RFQ 한 건 → 여섯 에이전트 → 검수된 제안서

1 **역할 분리**

SI 회사의 분업을 그대로 에이전트 6종으로 옮겼다

2 **추적 가능한 숫자**

모든 수치는 스크립트 출력과 가정 ID로 출처가 남는다

3 **독립 검수**

반려 루프가 과장과 누락을 잡아 고객 발송본의 품질을 지킨다

다음: 같은 골격으로 BOM 원가 절감 · 공급망 리스크 모니터링 에이전트 만들기